



Мануал по SQL



О нас

FAANG School – IT онлайн-школа с первоклассными образовательными продуктами, которая выпускает разработчиков с гибким мышлением и мощным стеком.

Наша миссия

Помогаем реализовать амбиции, стать профессиональными разработчиками и получить работу мечты в лучших IT-компаниях в кратчайшие сроки!

Что такое Java Буткемп?

Java Буткемп — это интенсивное обучение в формате командной работы над мощным проектом, где ты получишь максимально приближенный к реальному опыт для успешного трудоустройства с высокой зарплатой в кратчайшие сроки.

Для этого мы адаптировали практики разработки BigTech-компаний, внедрили передовые инструменты индустрии и организовали работу по гибкой методологии Scrum, чтобы ты выделялся среди других кандидатов на рынке.

**Стремишься стать востребованным разработчиком и получить работу мечты в кратчайшие сроки?
Скорее записывайся на Java Буткемп!**

[**ЗАПИСАТЬСЯ НА ЭКСКУРСИЮ**](#)



SQL

ОСНОВЫ SQL



SQL

SQL, или **Structured Query Language**, — это язык общения с базами данных. Он позволяет выбирать определенные данные и строить сложные отчеты. Сегодня SQL — это универсальный язык данных. Он используется практически во всех технологиях, которые обрабатывают данные.

Пример данных

COUNTRY				CITY				
id	name	population	area	id	name	country_id	population	rating
1	France	66600000	640680	1	Paris	1	2243000	5
2	Germany	80700000	357000	2	Berlin	2	3460000	3
...	...	...	...	...	...	...	...	...

ПОЛУЧЕНИЕ ДАННЫХ ИЗ ОДНОЙ ТАБЛИЦЫ

Получить все столбцы из таблицы country:

Устанавливает рабочую директорию (папку) для последующих инструкций.

```
SELECT *
FROM country;
```

Получить названия городов, отсортированные по столбцу rating в порядке возрастания:

```
SELECT name
FROM city
ORDER BY rating [ASC];
```

Получить столбцы id и name из таблицы city:

```
SELECT id, name
FROM city;
```

Получить названия городов, отсортированные по столбцу rating в порядке убывания:

```
SELECT name
FROM city
ORDER BY rating DESC;
```

АЛИАСЫ

ДЛЯ СТОЛБЦОВ

```
SELECT name AS city_name
FROM city;
```

ДЛЯ ТАБЛИЦ

```
SELECT co.name, ci.name
FROM city AS ci
JOIN country AS co
ON ci.country_id = co.id;
```

ФИЛЬТРАЦИЯ ВЫВОДА

ОПЕРАТОРЫ СРАВНЕНИЯ

Получить названия городов с рейтингом выше 3:

```
SELECT name
FROM city
WHERE rating > 3;
```

Получить названия городов, которые не являются ни Берлином, ни Мадридом:

```
SELECT name
FROM city
WHERE name != 'Berlin'
AND name != 'Madrid';
```

ТЕКСТОВЫЕ ОПЕРАТОРЫ

Получить названия городов, которые начинаются на 'P' или заканчиваются на 's':

```
SELECT name
FROM city
WHERE name LIKE 'P%'
AND name LIKE '%s';
```

Получить названия городов, которые содержат 'ublin' (например, Dublin в Ирландии или Lublin в Польше):

```
SELECT name
FROM city
WHERE name LIKE '_ublin';
```

ДРУГИЕ ОПЕРАТОРЫ

Получить названия городов с населением от 500 тыс. до 5 млн:

```
SELECT name
FROM city
WHERE population BETWEEN 500000 AND 5000000;
```

Получить названия городов, у которых указан рейтинг:

```
SELECT name
FROM city
WHERE rating IS NOT NULL;
```

Получить названия городов, которые находятся в странах с ID 1, 4, 7 или 8:

```
SELECT name
FROM city
WHERE country_id IN (1, 4, 7, 8);
```

ЗАПРОС К НЕСКОЛЬКИМ ТАБЛИЦАМ

INNER JOIN

JOIN (или явно **INNER JOIN**) возвращает строки, у которых совпадают значения в обеих таблицах.

```
SELECT city.name, country.name
FROM city
[INNER] JOIN country
ON city.country_id = country.id;
```

CITY			COUNTRY	
id	name	country_id	id	name
1	Paris	1	1	France
2	Berlin	2	2	Germany
3	Warsaw	4	3	Iceland

Вся база SQL для начинающих за 1 час

Всё, что вам надо знать о работе с SQL и даже больше, простыми словами рассказал главный преподаватель FAANG School — Влад Мишустин!

P. S. А также в видео узнаете о фиче, которая чаще всего вызывает трудности у начинающих разработчиков! Скорее смотреть!

[СМОТРЕТЬ ВИДЕО](#)



LEFT JOIN

LEFT JOIN возвращает все строки из левой таблицы с соответствующими строками из правой таблицы. Если совпадающих строк нет, возвращаются **NULLS** в качестве значений из второй таблицы.

```
SELECT city.name, country.name
FROM city
LEFT JOIN country
ON city.country_id = country.id;
```

CITY			COUNTRY	
id	name	country_id	id	name
1	Paris	1	1	France
2	Berlin	2	2	Germany
3	Warsaw	4	NULL	NULL

RIGHT JOIN

RIGHT JOIN возвращает все строки из правой таблицы с соответствующими строками из левой таблицы. Если совпадающих строк нет, возвращаются **NULLS** в качестве значений из первой таблицы.

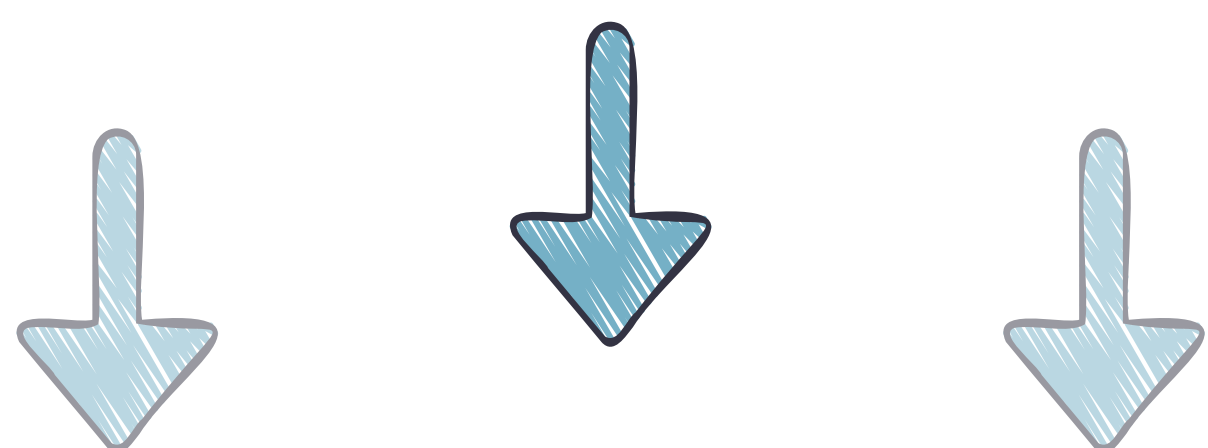
```
SELECT city.name, country.name
FROM city
RIGHT JOIN country
ON city.country_id = country.id;
```

CITY			COUNTRY	
id	name	country_id	id	name
1	Paris	1	1	France
2	Berlin	2	2	Germany
NULL	NULL	NULL	3	Iceland

FULL JOIN

FULL JOIN (или явно **FULL OUTER JOIN**) возвращает все строки из обеих таблиц — если совпадающей строки во второй таблице нет, возвращаются **NULLS**.

```
SELECT city.name, country.name
FROM city
FULL [OUTER] JOIN country
ON city.country_id = country.id;
```



CITY			COUNTRY	
id	name	country_id	id	name
1	Paris	1	1	France
2	Berlin	2	2	Germany
3	Warsaw	4	NULL	NULL
NULL	NULL	NULL	3	Iceland

CROSS JOIN

CROSS JOIN возвращает все возможные комбинации строк из обеих таблиц. Существует два ВОЗМОЖНЫХ СИНТАКСИСА.

```
SELECT city.name, country.name
FROM city
CROSS JOIN country;

SELECT city.name, country.name
FROM city, country;
```

CITY			COUNTRY	
id	name	country_id	id	name
1	Paris	1	1	France
1	Paris	1	2	Germany
2	Berlin	2	1	France
2	Berlin	2	2	Germany

NATURAL JOIN

NATURAL JOIN объединяет таблицы по всем колонкам с одинаковым именем.

```
SELECT city.name, country.name
FROM city
NATURAL JOIN country;
```

CITY			COUNTRY	
country_id	id	name	name	id
6	6	San Marino	San Marino	6
7	7	Vatican City	Vatican City	7
5	9	Greece	Greece	9
10	11	Monaco	Monaco	10

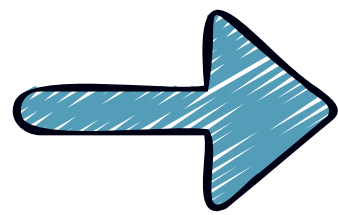
NATURAL JOIN использует следующие колонки для сопоставления строк: **city.id, city.name, country.id, country.name**.

NATURAL JOIN крайне редко используется на практике.

АГРЕГАЦИЯ И ГРУППИРОВКА

GROUP BY используется для группировки строк с одинаковыми значениями в одну или несколько групп. Он вычисляет сводки (агрегаты) для каждой уникальной комбинации значений

CITY		
id	name	country_id
1	Paris	1
101	Marseille	1
102	Lyon	1
2	Berlin	2
103	Hamburg	2
104	Munich	2
3	Warsaw	4
105	Cracow	4



CITY	
country_id	count
1	3
2	3
4	2

АГРЕГАЦИОННЫЕ ФУНКЦИИ

- **avg**(expr) — среднее значение для строк в группе
- **count**(expr) — количество значений для строк в группе
- **max**(expr) — максимальное значение в группе
- **min**(expr) — минимальное значение в группе
- **sum**(expr) — сумма значений в группе

ПРИМЕРЫ ЗАПРОСОВ

Узнать количество городов:

```
SELECT COUNT(*)
FROM city;
```

Узнать количество уникальных значений country_id:

```
SELECT COUNT(DISTINCT country_id)
FROM city;
```

Узнать общее население городов в соответствующих странах:

```
SELECT country_id, SUM(population)
FROM city
GROUP BY country_id;
```

Узнать количество городов с ненулевыми рейтингами:

```
SELECT COUNT(rating)
FROM city;
```

Узнать наименьшее и наибольшее население стран:

```
SELECT MIN(population), MAX(population)
FROM country;
```

Узнать средний рейтинг городов по country_id, если среднее выше 3.0:

```
SELECT country_id, AVG(population)
FROM city
GROUP BY country_id
HAVING AVG(rating) > 3.0;
```

ПОДЗАПРОСЫ

Подзапрос — это запрос, который вложен в другой запрос или внутри другого подзапроса. Существуют разные типы подзапросов.

ЕДИНИЧНОЕ ЗНАЧЕНИЕ

Самый простой подзапрос возвращает ровно один столбец и ровно одну строку. Он может использоваться с операторами сравнения $<$, $<=$, $>$, $>=$.

Этот запрос находит города с таким же рейтингом, как у Парижа:

```
SELECT name FROM city
WHERE rating = (
  SELECT rating
  FROM city
  WHERE name = 'Paris'
);
```

МНОЖЕСТВЕННЫЕ ЗНАЧЕНИЯ

Подзапрос может вернуть несколько столбцов или несколько строк. Такие подзапросы могут использоваться с операторами IN, EXISTS, ALL или ANY.

Этот запрос находит города в странах, где население превышает 20 миллионов:

```
SELECT name
FROM city
WHERE country_id IN (
  SELECT country_id
  FROM country
  WHERE population > 20000000
);
```

СВЯЗАННЫЕ

Связанный подзапрос ссылается на таблицы, представленные в внешнем запросе. Связанный подзапрос зависит от внешнего запроса. Его нельзя выполнить независимо от внешнего запроса.

Этот запрос находит города с населением больше среднего населения в стране:

```
SELECT *
FROM city main_city
WHERE population > (
  SELECT AVG(population)
  FROM city average_city
  WHERE average_city.country_id =
    main_city.country_id
);
```

Этот запрос находит страны, в которых есть хотя бы один город:

```
SELECT name
FROM country
WHERE EXISTS (
  SELECT *
  FROM city
  WHERE country_id = country.id
);
```

ОПЕРАЦИИ МНОЖЕСТВ

Операции с множествами используются для объединения результатов двух или более запросов в один результат. Объединенные запросы должны возвращать одинаковое количество столбцов и сопоставимые типы данных. Имена соответствующих столбцов могут различаться.

CYCLING			SKATING		
id	name	country	id	name	country
1	YK	DE	1	YK	DE
2	ZG	DE	2	DF	DE
3	WT	PL	3	AK	PL
...	...	...	...	...	...

UNION

UNION объединяет результаты двух наборов результатов и удаляет дубликаты. UNION ALL не удаляет дубликаты.

Этот запрос отображает немецких велосипедистов вместе с немецкими фигуристами:

```
SELECT name
FROM cycling
WHERE country = 'DE'
UNION / UNION ALL
SELECT name
FROM skating
WHERE country = 'DE';
```

INTERSECT

INTERSECT возвращает только строки, которые появляются в обоих наборах результатов

Этот запрос отображает немецких велосипедистов, которые также являются немецкими фигуристами в одно и то же время:

```
SELECT name
FROM cycling
WHERE country = 'DE'
INTERSECT
SELECT name
FROM skating
WHERE country = 'DE';
```

EXCEPT

EXCEPT возвращает только строки, которые появляются в первом результате, но не появляются во втором результате.

Этот запрос отображает немецких велосипедистов, которые также являются немецкими фигуристами в одно и то же время:

```
SELECT name
FROM cycling
WHERE country = 'DE'
EXCEPT / MINUS
SELECT name
FROM skating
WHERE country = 'DE';
```

Думаешь, что сейчас в IT не попасть?

Потребность в квалифицированных специалистах постоянно растет. Ждем тебя на бесплатной консультации, где развеем все мифы о старте в индустрии и дадим индивидуальные рекомендации для твоего развития в IT-сфере!

[УЗНАТЬ ПОДРОБНЕЕ](#)

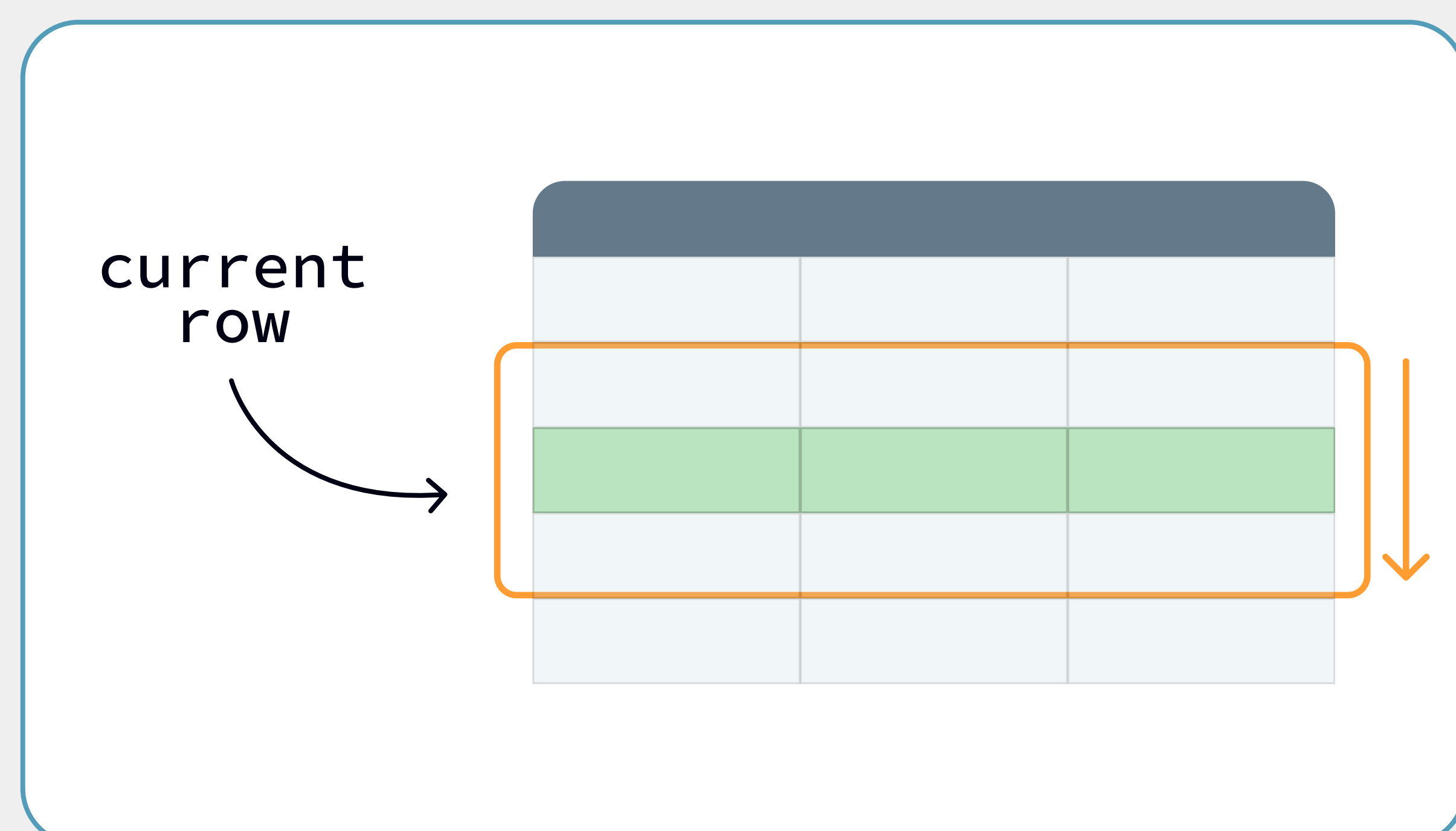


ОСНОВНЫЕ ФУНКЦИИ SQL



ОКОННЫЕ ФУНКЦИИ

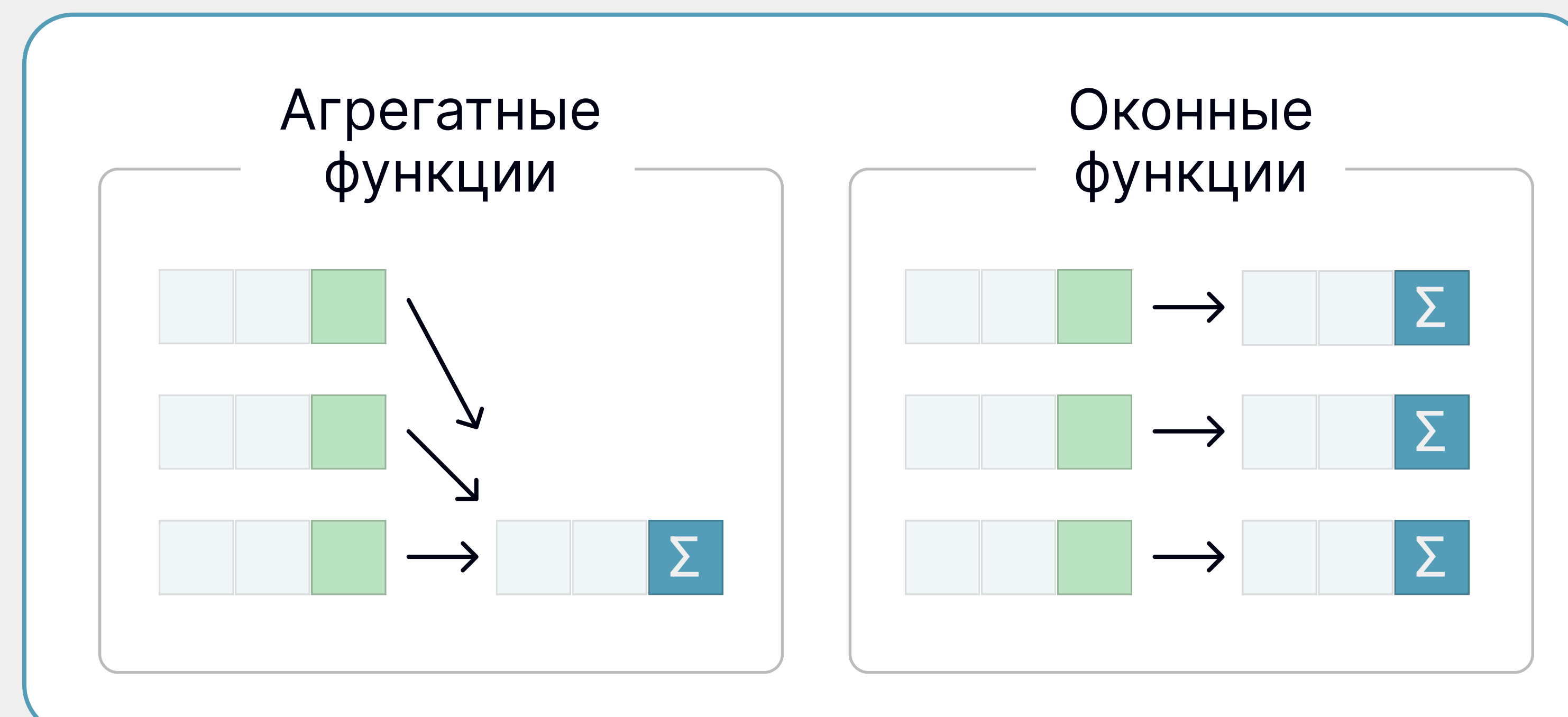
Оконные функции вычисляют свой результат на основе скользящего окна, набора строк, которые каким-то образом связаны с текущей строкой.



АГРЕГАЦИОННЫЕ ФУНКЦИИ VS. ОКОННЫЕ ФУНКЦИИ

Агрегатные функции сводят несколько строк в одну, возвращая одно значение для группы строк.

Оконные функции не убирают строки и не изменяют их количество.



СИНТАКСИС

```
SELECT city, month,
       sum(sold) OVER (
         PARTITION BY city
         ORDER BY month
         RANGE UNBOUNDED PRECEDING) total
FROM sales;
```

```
SELECT <column_1>, <column_2>,
       <window_function>() OVER (
         PARTITION BY <...>
         ORDER BY <...>
         <window_frame>) <window_column_alias>
FROM <table_name>;
```

ОПРЕДЕЛЕНИЕ ИМЕНИ ОКНА

```
SELECT country, city,
       rank() OVER country_sold_avg
FROM sales
WHERE month BETWEEN 1 AND 6
GROUP BY country, city
HAVING sum(sold) > 10000
WINDOW country_sold_avg AS (
  PARTITION BY country
  ORDER BY Y avg(sold) DESC)
GROUP BY country, city;
```

```
SELECT <column_1>, <column_2>,
       <window_function>() OVER <window_name>
FROM <table_name>
WHERE <...>
GROUP BY <...>
HAVING <...>
WINDOW <window_name> AS (
  PARTITION BY <...>
  ORDER BY <...>
  GROUP BY <...>;
```

PARTITION BY, ORDER BY и определение окна (window frame definition) — все они являются необязательными.

ЛОГИЧЕСКИЙ ПОРЯДОК ОПЕРАЦИЙ В SQL

1. FROM, JOIN
2. WHERE
3. GROUP BY
4. aggregate functions
5. HAVING
6. **window functions**
7. SELECT
8. DISTINCT
9. UNION/INTERSECT/EXCEPT
10. ORDER BY
11. OFFSET
12. LIMIT/FETCH/TOP

Вы можете использовать оконные функции в SELECT и ORDER BY. Однако нельзя использовать оконные функции в FROM, WHERE, GROUP BY или HAVING.

PARTITION BY

Разделяет строки на несколько групп, называемых разделами (partitions), к которым применяется оконная функция.

month	city	sold
1	Rome	200
2	Paris	500
1	London	100
1	Paris	300
2	Rome	300
2	London	400
3	Rome	400

PARTITION BY CITY			
month	city	sold	sold
1	Paris	300	800
2	Paris	500	800
1	Rome	200	900
2	Rome	300	900
3	Rome	400	900
1	London	100	500
2	London	400	500

По умолчанию: без PARTITION BY, весь результат является одним разделом.

ORDER BY

Указывает порядок строк в каждом разделе, к которым применяется оконная функция.

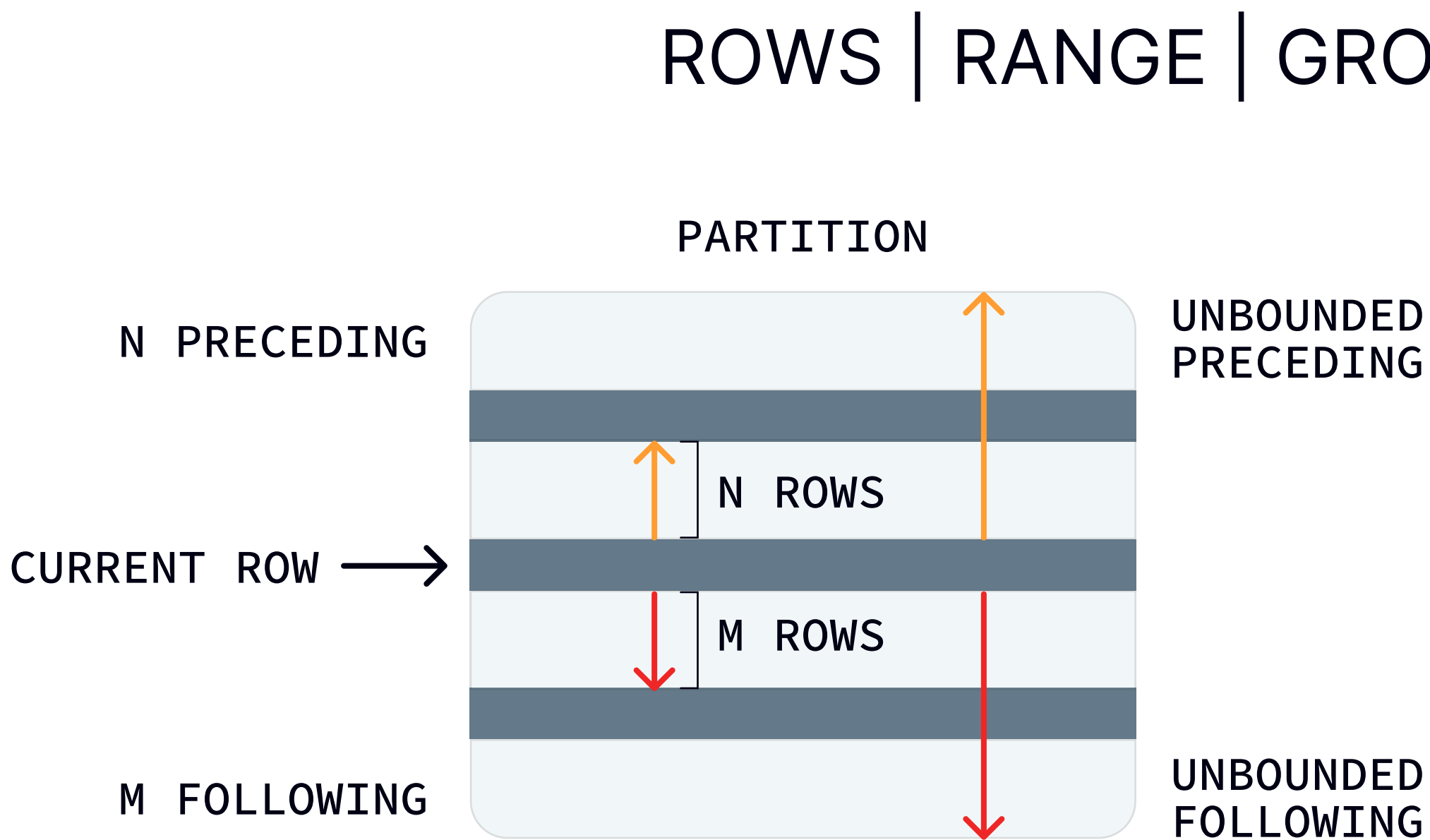
sold	city	month
200	Rome	1
500	Paris	2
100	London	1
300	Paris	1
300	Rome	2
400	London	2
400	Rome	3

PARTITION BY CITY ORDER BY MONTH		
sold	city	sold
300	Paris	1
500	Paris	2
200	Rome	1
300	Rome	2
400	Rome	3
100	London	1
400	London	2

По умолчанию: без ORDER BY, порядок строк внутри каждого раздела произволен.

ОКОННАЯ РАМКА

Оконная рамка — это набор строк, которые каким-то образом связаны с текущей строкой. Оконная рамка оценивается отдельно в каждом разделе.



Границы могут быть любыми из следующих опций:

- UNBOUNDED PRECEDING
- n PRECEDING
- CURRENT ROW
- n FOLLOWING
- UNBOUNDED FOLLOWING

Значение lower_bound должно быть ДО upper_bound.

ROWS BETWEEN
1 PRECEDING AND
1 FOLLOWING

	city	sold	month
	Paris	300	1
	Rome	200	1
	Paris	500	2
	Rome	100	4
↑	Paris	200	4
↑	Paris	300	5
↑	Rome	200	5
↑	London	200	5
↑	London	100	6
↑	Rome	300	6

1 row before the current row
and 1 row after the current
row

RANGE BETWEEN
1 PRECEDING AND
1 FOLLOWING

	city	sold	month
	Paris	300	1
	Rome	200	1
	Paris	500	2
	Rome	100	4
↑	Paris	200	4
↑	Paris	300	5
↑	Rome	200	5
↑	London	200	5
↑	London	100	6
↑	Rome	300	6

values in the range between
3 and 5 ORDER BY must
contain a single expression

GROUPS BETWEEN
1 PRECEDING AND
1 FOLLOWING

	city	sold	month
	Paris	300	1
	Rome	200	1
	Paris	500	2
	Rome	100	4
↑	Paris	200	4
↑	Paris	300	5
↑	Rome	200	5
↑	London	200	5
↑	London	100	6
↑	Rome	300	6

1 group before the current row
and 1 group after the current
row regardless of the value

As of 2020, GROUPS is only supported in PostgreSQL 11 and up.

АББРЕВИАТУРА

аббревиатура	значение
UNBOUNDED PRECEDING	Между началом и текущей строкой
n PRECEDING	Между n строками до и текущей строкой
CURRENT ROW	Текущая строка
n FOLLOWING	Между текущей строкой и n следующими
UNBOUNDED FOLLOWING	Между текущей строкой и концом

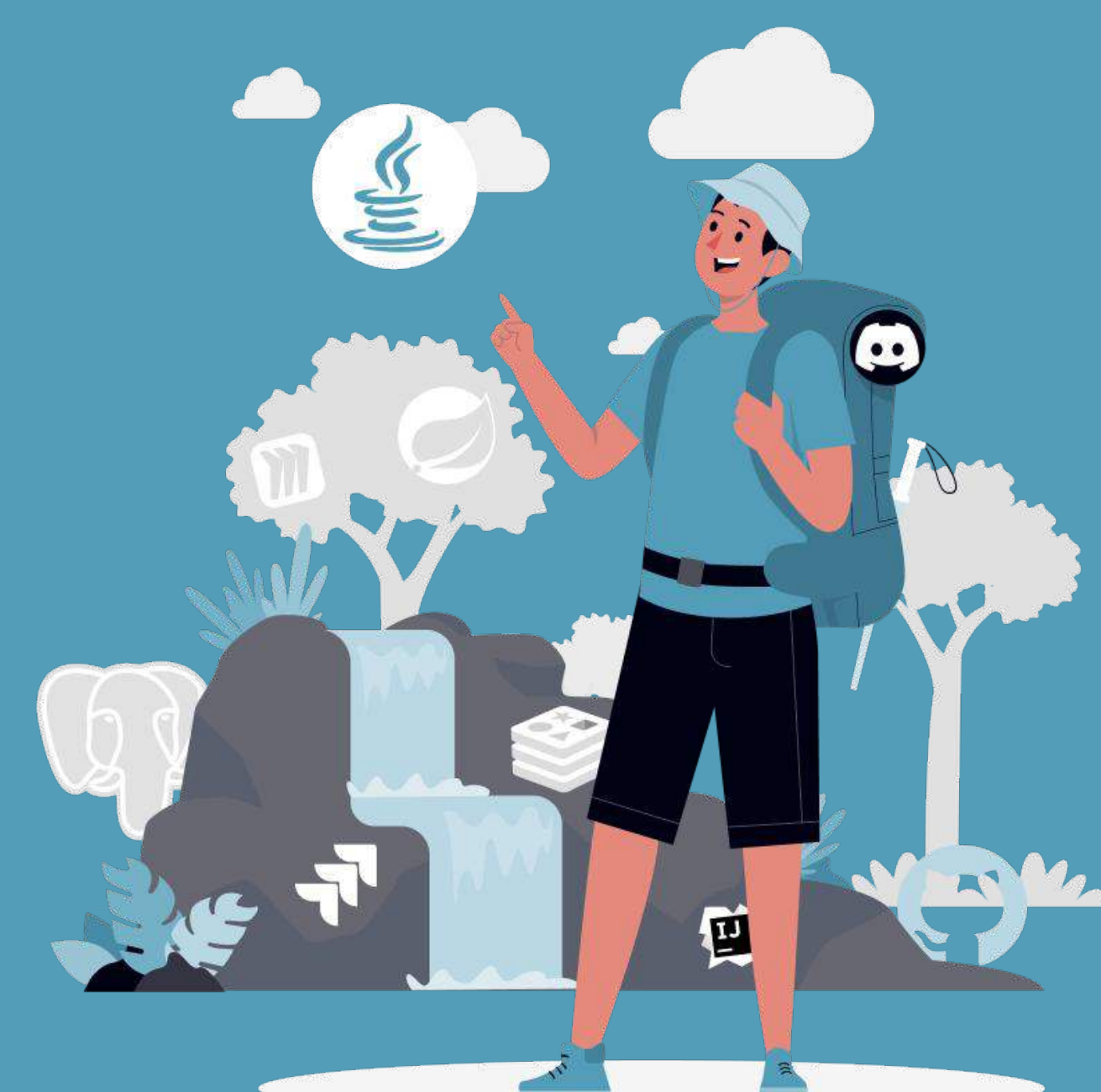
ДЕФОЛТНАЯ ОКОННАЯ РАМКА

- Если указано **ORDER BY**, то спецификация окна — это RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW.
- Без **ORDER BY** рамка — это RANGE BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING.

Почувствуй себя студентом FAANG School

Запишись на интерактивную экскурсию по Java Буткемпу!

[ЗАПИСАТЬСЯ НА ЭКСКУРСИЮ](#)



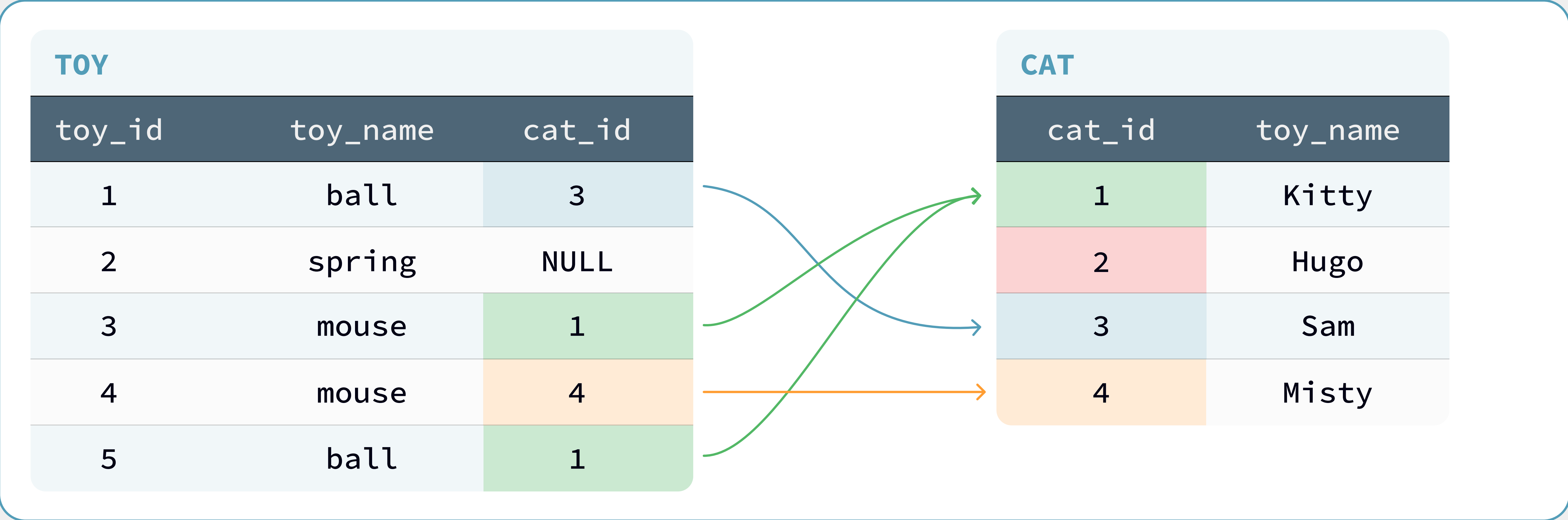


СБОРКА ПО SQL JOIN



СОЕДИНЕНИЕ ТАБЛИЦ

JOIN объединяет данные из двух таблиц.



JOIN обычно объединяет строки с одинаковыми значениями для указанных столбцов. Обычно одна таблица содержит первичный ключ, который представляет уникальные значения для этой таблицы (например, столбец `cat_id` в таблице `CAT`). Другая таблица имеет столбец или столбцы, которые ссылаются на первичные ключи в первой таблице (например, столбец `cat_id` в таблице `TOY`). Такие столбцы называются внешними ключами. Условие JOIN — это равенство между первичными ключами одной таблицы и столбцами, ссылающимися на них, в другой таблице

JOIN

JOIN возвращает все строки, которые соответствуют условию ON. Также называется INNER JOIN.

```
SELECT *
FROM toy
JOIN cat
ON toy.cat_id = cat.cat_id;
```

toy_id	toy_name	cat_id	cat_id	cat_name
5	ball	1	1	Kitty
3	mouse	1	1	Kitty
1	ball	3	3	Sam
4	mouse	4	4	Misty

Существует другой, более старый синтаксис, но он **не рекомендуется**.

```
SELECT *
FROM toy, cat
WHERE toy.cat_id = cat.cat_id;
```

УСЛОВИЯ JOIN

Условие JOIN не обязательно должно быть равенством — это может быть любое условие, которое вам нужно. JOIN не интерпретирует условие JOIN, он просто проверяет, удовлетворяют ли строки данному условию.

Чтобы сослаться на столбец в условии JOIN, вы должны использовать полное имя столбца: сначала имя таблицы, затем точка (.) и имя столбца.

```
ON cat.cat_id = toy.cat_id
```

LEFT JOIN

LEFT JOIN возвращает все строки из левой таблицы с соответствующими строками из правой таблицы. Строки без соответствий заполняются NULL. LEFT JOIN также называется LEFT OUTER JOIN.

```
SELECT *
FROM toy
LEFT JOIN cat
ON toy.cat_id = cat.cat_id;
```

toy_id	toy_name	cat_id	cat_id	cat_name
5	ball	1	1	Kitty
3	mouse	1	1	Kitty
1	ball	3	3	Sam
4	mouse	4	4	Misty
2	spring	NULL	NULL	NULL

whole left table

RIGHT JOIN

RIGHT JOIN возвращает все строки из правой таблицы с соответствующими строками из левой таблицы. Строки без соответствий заполняются NULL. RIGHT JOIN также называется RIGHT OUTER JOIN.

```
SELECT *
FROM toy
RIGHT JOIN cat
ON toy.cat_id = cat.cat_id;
```

toy_id	toy_name	cat_id	cat_id	cat_name
5	ball	1	1	Kitty
3	mouse	1	1	Kitty
NULL	NULL	NULL	2	Hugo
1	ball	3	3	Sam
4	mouse	4	4	Misty

whole right table

FULL JOIN

FULL JOIN возвращает все строки из левой таблицы и все строки из правой таблицы. Он заполняет строки без соответствий NULL. FULL JOIN также называется FULL OUTER JOIN.

```
SELECT *
FROM toy
FULL JOIN cat
ON toy.cat_id = cat.cat_id;
```

toy_id	toy_name	cat_id	cat_id	cat_name
5	ball	1	1	Kitty
3	mouse	1	1	Kitty
NULL	NULL	NULL	2	Hugo
1	ball	3	3	Sam
4	mouse	4	4	Misty
2	spring	NULL	NULL	NULL

whole left table

whole right table



Сводка по стандартным функциям SQL



ТЕКСТОВЫЕ ФУНКЦИИ

КОНКАТЕНАЦИЯ

Используйте оператор `||`, чтобы объединить две строки:

```
SELECT 'Hi ' || 'there!';  
-- result: Hi there!
```

Помните, что вы можете объединять только строковые значения с помощью оператора `||`. Используйте этот трюк для чисел:

```
SELECT ' ' || 4 || 2;  
-- result: 42
```

Некоторые базы данных реализуют не-стандартные решения для объединения строк, такие как `CONCAT()` или `CONCAT_WS()`. Проверьте документацию для вашей конкретной базы данных.

ОПЕРАТОР LIKE — СООТВЕТСТВИЕ ПО ЗАДАННОМУ ШАБЛОНУ

Используйте символ `_`, чтобы заменить один любой символ. Используйте `%`, чтобы заменить любое количество символов (включая 0 символов).

Выберите все имена, которые начинаются с любой буквы, за которой следует 'atherine':

```
SELECT name  
FROM names  
WHERE name LIKE '_atherine';
```

Выбрать все имена, которые заканчиваются на 'a':

```
SELECT name  
FROM names  
WHERE name LIKE '%a';
```

ПОЛЕЗНЫЕ ФУНКЦИИ

Получить количество символов в строке:

```
SELECT LENGTH('LearnSQL.com');  
-- result: 12
```

Преобразовать все буквы в строчные:

```
SELECT LOWER('LEARNSQL.COM');  
-- result: learnsql.com
```

Преобразовать все буквы в прописные:

```
SELECT UPPER('LearnSQL.com');  
-- result: LEARNSQL.COM
```

Преобразовать все буквы в строчные и каждую первую букву в прописную (не реализовано в MySQL и SQL Server)

```
SELECT INITCAP('edgar frank ted codd');  
-- result: Edgar Frank Ted Codd
```

Получить часть строки:

```
SELECT SUBSTRING('LearnSQL.com', 9);  
-- result: .com  
SELECT SUBSTRING('LearnSQL.com', 0, 6);  
-- result: Learn
```

Заменить часть строки:

```
SELECT REPLACE('LearnSQL.com', 'SQL', 'Python');  
-- result: LearnPython.com
```

ЧИСЛОВЫЕ ФУНКЦИИ

БАЗОВЫЕ ОПЕРАЦИИ

Используйте +, -, *, / для выполнения базовой математики.

```
SELECT 60 * 60 * 24 * 7;
-- result: 604800
```

ПРЕОБРАЗОВАНИЕ

Иногда необходимо изменить тип числа. Функция CAST() помогает вам изменить значение почти в любой тип (integer, numeric, double precision, float, char, varchar, и многие другие).

Получите число в виде целого числа (без округления):

```
SELECT CAST(1234.567 AS integer);
-- result: 1234
```

Измените тип столбца на двойную точность.

```
SELECT CAST(column AS double precision);
```

NULL

Чтобы получить все строки с отсутствующим значением в столбце price:

```
WHERE price IS NULL
```

Чтобы получить все строки с заполненным столбцом weight:

```
WHERE weight IS NOT NULL
```

Почему не следует использовать price = NULL или weight != NULL? Потому что базы данных не знают, являются ли эти выражения истинными или ложными — они оцениваются как NULL. Кроме того, если вы используете функцию или конкатенацию на столбце, который содержит NULL в некоторых строках, это значение будет передано дальше.

domain	LENGTH(domain)
LearnSQL.com	12
LearnPython.com	15
NULL	NULL
vertabelo.com	13

ПОЛЕЗНЫЕ ФУНКЦИИ COALESCE(x, y, ...)

Чтобы заменить NULL в запросе чем-то значимым:

```
SELECT
  domain,
  COALESCE( domain, 'domain missing')
FROM contacts;
```

domain	coalesce
LearnSQL.com	LearnSQL.com
NULL	domain missing

Функция COALESCE() принимает любое количество аргументов и возвращает значение первого аргумента, который не является NULL.

CASE WHEN

Базовая версия CASE WHEN проверяет, равны ли значения, и возвращает результат.

Базовая версия CASE WHEN проверяет, равны ли значения (например, если fee равно 50, то возвращается 'normal'). Если в CASE WHEN нет подходящего значения, будет возвращено значение ELSE (например, если fee равно 49, то отобразится 'not available').

```
SELECT
CASE fee
  WHEN 50 THEN 'normal'
  WHEN 10 THEN 'reduced'
  WHEN 0 THEN 'free'
  ELSE 'not available'
END AS tariff
FROM ticket_types;
```

Самый популярный тип – это искомый CASE WHEN – он позволяет задавать условия (как если бы вы писали их в предложении WHERE), оценивает их по порядку и возвращает значение для первого удовлетворенного условия.

```
SELECT
CASE
  WHEN score >= 90 THEN 'A'
  WHEN score > 60 THEN 'B'
  ELSE 'F'
END AS grade
FROM test_results;
```

Здесь все студенты, набравшие не менее 90 баллов, получают оценку A, те, кто набрал выше 60 (и ниже 90), получают оценку B, а остальные получают оценку F.

АГРЕГАЦИЯ И ГРУППИРОВАНИЕ

- **COUNT**(expr) — количество строк в группе
- **SUM**(expr) — сумма значений в группе
- **AVG**(expr) — среднее значение для строк в группе
- **MIN**(expr) — минимальное значение в группе
- **MAX**(expr) — максимальное значение в группе

Чтобы получить количество строк в таблице:

```
SELECT COUNT(*)
FROM city;
```

Чтобы получить количество уникальных значений в столбце:

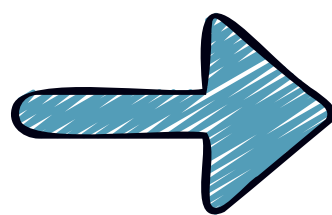
```
SELECT COUNT(rating)
FROM city;
```

Чтобы получить количество уникальных значений в столбце:

```
SELECT COUNT(DISTINCT country_id)
FROM city;
```

GROUP BY

CITY	
name	country_id
Paris	1
Marseille	1
Lyon	1
Berlin	2
Hamburg	2
Munich	2
Warsaw	4
Cracow	4



CITY	
country_id	count
1	3
2	3
4	2

Пример выше — количество городов в каждой стране:

```
SELECT name, COUNT(country_id)
FROM city
GROUP BY name;
```

Чтобы получить все строки с заполненным столбцом weight:

```
SELECT city_id, AVG(rating)
FROM ratings
GROUP BY city_id;
```

Распространенная ошибка: COUNT (*) и LEFT JOIN

Когда вы соединяете таблицы следующим образом: client LEFT JOIN project, и хотите получить количество проектов для каждого клиента, COUNT (*) вернет 1 для каждого клиента, даже если вы никогда с ними не работали. Это происходит потому, что они все равно присутствуют в списке, но с NULL в полях, связанных с проектом, после JOIN. Чтобы получить правильное количество (0 для клиентов, с которыми вы никогда не работали), нужно считать значения в колонке другой таблицы, например, COUNT (project_name).

Проверь свои знания в Java!

Пройди тест и узнай, как с твоим уровнем в Java стать востребованным программистом.
Бонус — Roadmap по Java

[ПРОЙТИ ТЕСТ](#)



ДАТА И ВРЕМЯ

Существует 3 основных типа времени: **date**, **time** и **timestamp**. Время выражается в 24-часовом формате и может включать доли секунд и временную зону (например, 14:39:53.662522-05).

2021-12-31 14:39:53.662522-05



YYYY-mm-dd HH:MM:SS.ssssss±TZ

14:39:53.662522-05 — это почти 2:40 дня по центральному времени (например, в Чикаго; в UTC это будет 7:40 вечера). Буквы в приведенном примере обозначают:

ФОРМАТ:

Год: YYYY

Месяц: MM

День: DD

Часы: HH (24-часовой формат)

Минуты: MI

Секунды: SS

Миллисекунды: ssssss (опционально)

В ЧАСТИ ВРЕМЕНИ:

HH — час, дополненный нулями, в 24-часовом формате.

MM — минуты.

SS — секунды. Необязательно.

sssss — дробные доли секунд, которые могут быть выражены от 1 до 6 цифр. Необязательно.

±TZ — часовой пояс. Он должен начинаться с символа + или -, и использовать две цифры относительно UTC. Необязательно.

ЧТОБЫ УЗНАТЬ ТЕКУЩЕЕ ВРЕМЯ В SQL, ВЫ МОЖЕТЕ ИСПОЛЬЗОВАТЬ:

CURRENT_TIME — найти время в данный момент.

CURRENT_DATE — получить сегодняшнюю дату.

CURRENT_TIMESTAMP — получить метку времени с датой и временем.

Чтобы создать date, time или timestamp, просто запишите значение в виде строки и приведите его к нужному типу.

```
SELECT CAST('2021-12-31' AS date);
SELECT CAST('15:31' AS time);
SELECT CAST('2021-12-31 23:59:29+02' AS timestamp);
SELECT CAST('15:31.124769' AS time);
```

Будьте внимательны с последним примером — он будет интерпретирован как 15 минут 31 секунда и 124769 микросекунд! Всегда рекомендуется указывать 00: явно для часов: '00:15:31.124769'.

Вы можете пропустить приведение типов в простых условиях — база данных поймет, что вы имеете в виду.

```
SELECT airline, flight_number, departure_time
FROM airport_schedule
WHERE departure_time < '12:00';
```

ИНТЕРВАЛЫ

В SQL Server интервалы не реализованы — используйте функции DATEDIFF() и DATEADD().

Чтобы получить самый простой интервал, вычтите одно значение времени из другого:

```
SELECT CAST('2021-12-31 23:59:59' AS timestamp) - CAST('2021-06-01 12:00:00' AS timestamp);
-- result: 213 days 11:59:59
```

Пример простейшего интервала: **INTERVAL '1' DAY**

Для определения интервала используйте ключевое слово INTERVAL, затем значение и ключевое слово времени (например, YEAR, MONTH, DAY, HOUR, MINUTE).

INTERVAL '1' YEAR + INTERVAL '3' MONTH

Syntax	What it does
INTERVAL 'x-y' YEAR TO MONTH	INTERVAL 'x year y month'
INTERVAL 'x-y' DAY TO SECOND	INTERVAL 'x day y second'

В MySQL используйте `year\_month` вместо `YEAR TO MONTH` и `day\_second` вместо `DAY TO SECOND`.

Чтобы получить последний день месяца, добавьте один месяц и вычтите один день:

```
SELECT CAST('2021-02-01' AS date)
+ INTERVAL '1' MONTH
- INTERVAL '1' D DAY;
```

Чтобы получить все события на следующие три месяца, начиная с сегодняшнего дня:

```
SELECT event_date, event_name
FROM calendar
WHERE event_date BETWEEN CURRENT_DATE AND CURRENT_DATE + INTERVAL '3' MONTH;
```

Чтобы получить часть даты:

```
SELECT EXTRACT(YEAR FROM birthday)
FROM artists;
```

Одно из возможных возвращаемых значений: 1946. В SQL Server используйте функцию DATEPART(part, date).



Хочешь узнать больше о том,
как именно тебе начать
карьеру в IT в 2024?

Переходи по ссылке и получай
БЕСПЛАТНУЮ карьерную
консультацию от команды
FAANG School!

[ЗАПИСАТЬСЯ](#)

Подписывайся на нас
в соц.сетях:

 [Telegram](#)

 [YouTube](#)

 [ВК](#)

